When inserting a `Post` entity:

```
doInJPA(entityManager -> {
    Post post = new Post();
    post.setTitle("High-Performance Java Persistence");
    entityManager.persist(post);
});
printCacheRegionStatistics(Post.class.getName());
```

Hibernate is going to generate the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {}
```

> Because it supports write-through `READ_WRITE` entity caching, the sequence generator is preferred over identity columns. The behavior might change in future, so it is better to check the HHH-7964[a] JIRA issue status.
>
> [a] https://hibernate.atlassian.net/browse/HHH-7964

15.5.6.3.2 Updating READ_WRITE cache entries

As already mentioned, the `READ_WRITE` cache concurrency strategy employs a soft locking mechanism to ensure data integrity.

1. The Hibernate transaction commit procedure triggers a `Session` flush.
2. The `EntityUpdateAction` replaces the current cache entry with a `Lock` object.
3. The `update` method is used for synchronous strategies. Therefore, it is a no-op in this case.
4. The after transaction callbacks are called, and the `EntityUpdateAction` executes the `afterUpdate` method of the `EntityRegionAccessStrategy`.
5. The `ReadWriteEhcacheEntityRegionAccessStrategy` replaces the `Lock` entry with an actual `Item`, encapsulating the entity disassembled state.

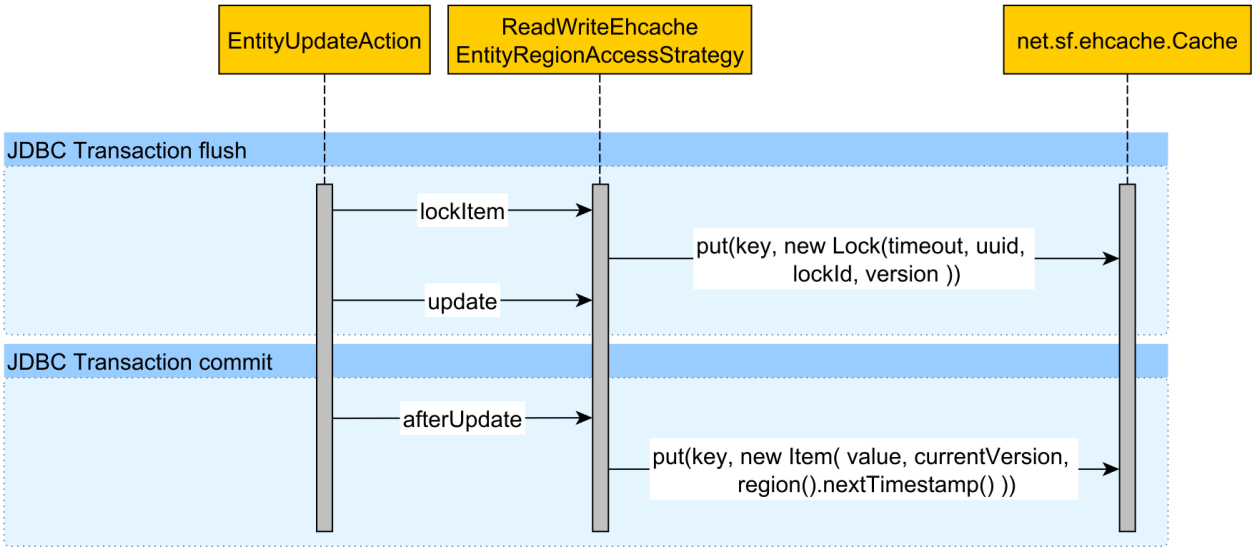

Figure 15.12: READ_WRITE update flow

Just like with database transactions, changes are applied directly, and locks are used to prevent other concurrent transactions from reading uncommitted data. When reading a `Lock` object from the cache, Hibernate knows that the associated entry is being modified by an uncommitted transaction. Therefore, it reads the entity from the database.

To visualize the whole process, when running the following test case:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    post.setTitle("High-Performance Hibernate");
    PostComment comment = post.getComments().remove(0);
    comment.setPost(null);
    entityManager.flush();

    printCacheRegionStatistics(Post.class.getName());
    printCacheRegionStatistics(Post.class.getName() + ".comments");
    printCacheRegionStatistics(PostComment.class.getName());
    LOGGER.debug("Commit after flush");
});
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");
printCacheRegionStatistics(PostComment.class.getName());
```

Hibernate generates the following output:

```
UPDATE post
SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0

DELETE FROM post_comment WHERE id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=Lock Source-UUID:69c2fd51-11a3-43c1-9db2-91f30624ac74 Lock-ID:0}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=Lock Source-UUID:e75094c1-6bc2-43f3-87e3-1dcdf6bee083 Lock-ID:1}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=Lock Source-UUID:99aafdef-7816-43ee-909d-5f10ab759c60 Lock-ID:0,
          2=[ value = CacheEntry(PostComment)[1,Hibernate part review],
              version=null, timestamp=5992022222598145 ]}

--Commit after flush

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=2],
Entries: {1=[ value = CacheEntry(Post)[1,High-Performance Hibernate,1],
              version=1, timestamp=5992019884548096 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=Lock Source-UUID:db769a0a-d65a-4911-952e-1d0bb851ed8d Lock-ID:1}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=Lock Source-UUID:f357da6a-665e-40d1-84c0-760e450df421 Lock-ID:0,
          2=[ value = CacheEntry(PostComment)[1,Hibernate part review],
              version=null, timestamp=5992019884109825 ]}
```

Right after the Persistence Context is flushed, Hibernate executes the associated SQL statements and adds `Lock` objects into the cache entries associated with the currently modifying `Post` entity and `comments` collection, as well as for the deleting `PostComment` entity.

After the transaction is committed, the `Post` entity cache entry is replaced with an `Item` object containing the updated disassembled state. Since `READ_WRITE` collections are not write-through, the `comments` collection cache entry is still a `Lock` object even after commit. Since the `PostComment` entity has been deleted, its cache entry is represented by a `Lock` entry.

15.5.6.3.3 Deleting READ_WRITE cache entries

Deleting entities is similar to the update process, as we can see from the following sequence diagram:

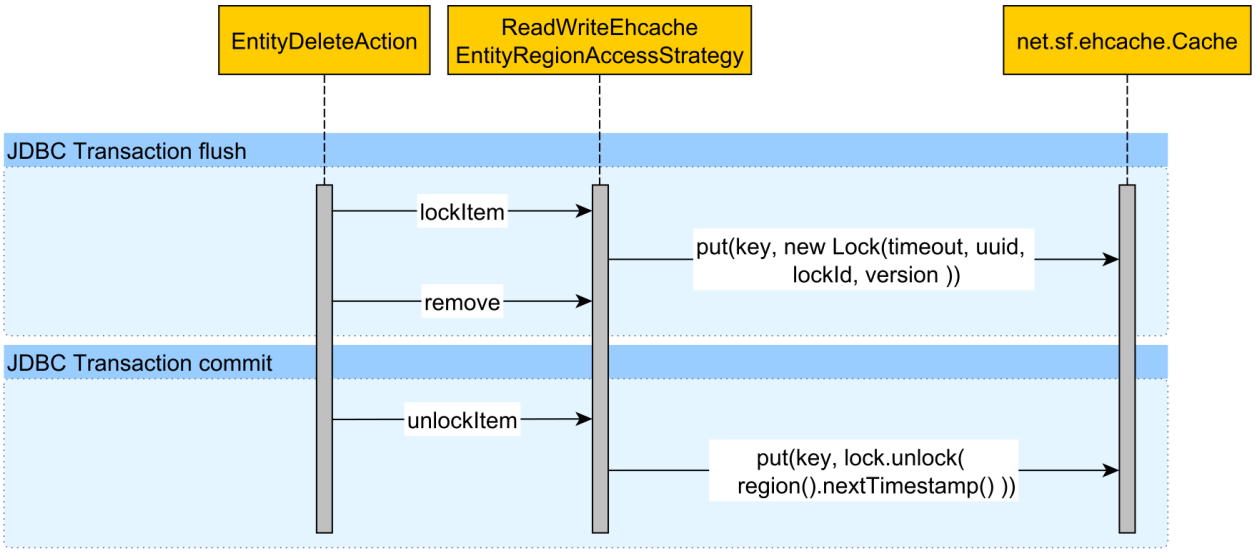

Figure 15.13: READ_WRITE delete flow

1. The Hibernate transaction commit procedure triggers a `Session` flush.
2. The `EntityDeleteAction` replaces the current cache entry with a `Lock` object
3. The `remove` method call doesn't do anything since `READ_WRITE` is an asynchronous cache concurrency strategy.
4. The after transaction callbacks are called, and the `EntityDeleteAction` executes the `unlockItem` method of the `EntityRegionAccessStrategy`.
5. The `ReadWriteEhcacheEntityRegionAccessStrategy` replaces the `Lock` entry with another `Lock` object whose timeout period is further increased.

After an entity is deleted, its associated second-level cache entry will be replaced by a `Lock` object, so that any subsequent request is redirected to reading from the database instead of using the second-level cache entry.

When running the same example used for deleting `NONSTRICT_READ_WRITE` cache entries, Hibernate generates the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=[ value = CacheEntry(Post)[1,High-Performance Java Persistence,0],
              version=0, timestamp=5992355751620608 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=[ value = CollectionCacheEntry[1,2],
              version=null, timestamp=5992355752042496 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=[ value = CacheEntry(PostComment)[1,JDBC part review],
              version=null, timestamp=5992355751624704 ],
          2=[ value = CacheEntry(PostComment)[1,Hibernate part review],
              version=null, timestamp=5992355751624705 ]}

DELETE FROM post_comment WHERE id = 1
DELETE FROM post_comment WHERE id = 2
DELETE FROM post WHERE id = 1 AND version = 0

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=1],
Entries: {1=Lock Source-UUID:b042192a-9ac6-4877-8663-018f898f1cdb Lock-ID:0}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {1=Lock Source-UUID:e75f034a-0346-4696-88fd-1b5100658a6f Lock-ID:1}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {1=Lock Source-UUID:1d13b830-e96d-40f5-aa2b-6a402fa6135d Lock-ID:0,
          2=Lock Source-UUID:1d13b830-e96d-40f5-aa2b-6a402fa6135d Lock-ID:1}
```

The delete operation does not remove entries from the second-level cache, but instead it replaces the previous `Item` entries with `Lock` objects. The next time a deleted cache entry is being read, Hibernate is going to redirect the request to the database, therefore guaranteeing strong consistency.

15.5.6.3.4 Soft locking concurrency control

Because the database is the system of record, strong consistency implies that uncommitted cache changes should not be read by other concurrent transactions. The `READ_WRITE` can store either an `Item` or a `Lock`.